



Comsats University Islamabad, Abbotabad Campus.

Student Information	Details
Student Name	Sami Ullah
Registration No.	SP24-BSE-156
Subject	Software Quality Engineering
Lab Report	Lab 1
Tasks Covered	Lab Task 1, Lab Task 2, Lab Task 3

1. Introduction

Software Quality Engineering (SQE) focuses on improving software quality by identifying errors, faults, failures, and defects through systematic testing. In these laboratory tasks, software is executed, test cases are designed, incorrect conditions are introduced, and the observed results are compared with the expected results.

2. Lab Task 1 – CourseResult Testing

Objective

To evaluate a simple Java program by observing its execution and by introducing errors, faults, and failures through test cases.

Problem Statement

Create a CourseResult class containing student name, course name, and grade. The display() method prints these values. Create CourseResult objects in a runner class and display their results.

Corrected Java Implementation

```
class CourseResult {
    public String studentName;
    public String courseName;
    public String grade;

    public void display() {
        System.out.println("Student Name is: " + studentName);
        System.out.println("Course Name is: " + courseName);
        System.out.println("Grade is: " + grade);
    }
}

public class CourseResultRun {
    public static void main(String[] args) {
        CourseResult c1 = new CourseResult();
        c1.studentName = "Ali";
        c1.courseName = "OOP";
        c1.grade = "A";
        c1.display();

        CourseResult c2 = new CourseResult();
        c2.studentName = "Saba";
        c2.courseName = "ICP";
        c2.grade = "A+";
        c2.display();
    }
}
```

Concept of Error, Fault and Failure

Term	Simple Meaning	Example in This Task
Error	A human mistake made while writing or changing the program.	Writing courseName incorrectly in the runner.
Fault	The incorrect code or defect caused by the error.	Using c1.coursename instead of c1.courseName.
Failure	The incorrect behavior observed when the faulty code executes.	The program displays null or incorrect course information.

Test Cases – Failures

TC ID	Test Input / Fault Introduced	Expected Result	Actual Result	Status
TC-1	studentName = "Ali", courseName = "OOP", grade = "A"	Ali, OOP, A should be displayed.	Course name is displayed as null after using the wrong variable name.	Fail
TC-2	grade = "AA" (invalid grade)	Invalid grade should be rejected or reported.	AA is displayed without validation.	Fail
TC-3	courseName = ""	Empty course name should be rejected or reported.	Blank course name is displayed.	Fail

Test Cases – Non-Failures

TC ID	Test Input	Expected Result	Actual Result	Status
TC-4	Ali, OOP, A	Student name, course and grade display correctly.	Correct values displayed.	Pass
TC-5	Saba, ICP, A+	Student name, course and grade display correctly.	Correct values displayed.	Pass
TC-6	Ahmed, DSA, B	Student name, course and grade display correctly.	Correct values displayed.	Pass

Task 1 Result

The correct implementation displays the entered CourseResult data. The introduced faults show that the program has no validation for empty or invalid grades, and a variable-name fault can cause incorrect output. Therefore, testing is required even for a small program.

3. Lab Task 2 – Point and Line Testing

Objective

To design Point and Line classes, calculate line length using two points, and test the functionality by introducing different faults.

Problem Statement

Create a Point class with x and y coordinates, an argument constructor, setters, getters, and a display function. Create a Line class containing startPoint and endPoint and calculate the line length.

Formula

Line length = $\sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2}$

Java Implementation

```
class Point {
    private double x;
    private double y;

    public Point(double x, double y) {
        this.x = x;
        this.y = y;
    }

    public void setX(double x) { this.x = x; }
    public void setY(double y) { this.y = y; }

    public double getX() { return x; }
    public double getY() { return y; }

    public void display() {
        System.out.println("(" + x + ", " + y + ")");
    }
}

class Line {
    private Point startPoint;
    private Point endPoint;

    public Line(Point startPoint, Point endPoint) {
        this.startPoint = startPoint;
        this.endPoint = endPoint;
    }

    public double length() {
        double dx = endPoint.getX() - startPoint.getX();
        double dy = endPoint.getY() - startPoint.getY();
        return Math.sqrt(dx * dx + dy * dy);
    }
}

public class LineRun {
    public static void main(String[] args) {
        Line line1 = new Line(new Point(0, 0), new Point(3, 4));
        Line line2 = new Line(new Point(1, 2), new Point(4, 6));

        System.out.println("Length of Line 1 = " + line1.length());
        System.out.println("Length of Line 2 = " + line2.length());
    }
}
```

Test Cases – Failures

TC ID	Fault Introduced	Expected Result	Actual Result	Status
TC-1	Use $dx = x_1 - x_2$ instead of $x_2 - x_1$.	Length should remain positive and correct because the difference is squared.	For this formula, result remains correct; no failure occurs.	Pass
TC-2	Use $dx * dx + dy$ instead of $dx * dx + dy * dy$.	Line from (0,0) to (3,4) should have length 5.	Incorrect length is produced.	Fail
TC-3	Swap start and end points without	Length should remain the same.	Length remains the same; no failure	Pass

	changing the formula.		occurs.	
--	-----------------------	--	---------	--

Note: The task asks for three failure and three non-failure cases. To clearly demonstrate failures, the following additional fault-oriented cases use inputs that expose the defect.

TC ID	Failure-Oriented Test	Expected Result	Actual Result	Status
F-1	Line (0,0) to (3,4), faulty formula omits dy^2 .	5.0	Incorrect value.	Fail
F-2	Line (2,2) to (2,2), faulty formula returns a fixed value 1.	0.0	1.0	Fail
F-3	Negative coordinate case: (-3,-4) to (0,0), faulty code uses Math.abs on only one component.	5.0	Incorrect value.	Fail

Test Cases – Non-Failures

TC ID	Test Input	Expected Result	Actual Result	Status
NF-1	(0,0) to (3,4)	Length = 5.0	5.0	Pass
NF-2	(1,1) to (1,6)	Length = 5.0	5.0	Pass
NF-3	(2,3) to (5,7)	Length = 5.0	5.0	Pass

Task 2 Result

The Line class correctly calculates distance when the Euclidean-distance formula is implemented. Faulty arithmetic, incorrect constants, or incomplete squaring can produce failures. Boundary and normal cases are useful for detecting such faults.

4. Lab Task 3 – CUOnline Defect Identification

Objective

To identify, document, and report defects found while testing a CUOnline-style university portal. The following are example defects suitable for a lab exercise and should be verified against the actual system before submission.

Defect Report

Defect ID	Module	Defect Description	Steps to Reproduce	Expected Result	Severity
DEF-01	Login	Invalid login credentials should produce a clear	Open login → enter invalid credentials → click Login.	A clear error message should appear without ambiguity.	Medium

		validation message.			
DEF-02	Course Registration	Registering without selecting a required course/option should be prevented.	Open registration → leave required field unselected → submit.	System should highlight the required field and prevent submission.	High
DEF-03	Profile	Invalid/empty required profile fields may be accepted without validation.	Open profile → remove a required value → save.	System should display validation and prevent invalid data.	Medium
DEF-04	Results	Result/grade information should remain consistent after refresh.	Open results → note displayed result → refresh page.	The same valid result should be displayed.	High
DEF-05	Navigation	A navigation link may lead to an incorrect or unavailable page.	Select the affected menu item.	The selected module should open successfully.	Medium

Defect Classification

Defect	Type	Priority	Recommended Action
DEF-01	Validation / Usability	Medium	Improve login validation and error feedback.
DEF-02	Functional	High	Add mandatory-field validation before submission.
DEF-03	Validation	Medium	Validate required profile fields.
DEF-04	Data / Functional	High	Verify result retrieval and state consistency.
DEF-05	Navigation / Functional	Medium	Verify route and page availability.

5. Overall Testing Summary

Lab	Main Activity	Testing Outcome
Lab 1	CourseResult execution and fault-based test cases	Correct cases passed; introduced validation and variable faults caused failures.
Lab 2	Point/Line implementation and length testing	Correct distance calculations passed; faulty formulas produced failures.
Lab 3	CUOnline defect identification	Defects were documented using

		reproducible steps, expected behavior and severity.
--	--	---

6. Conclusion

These laboratory tasks demonstrate the importance of software testing in Software Quality Engineering. An error is a human mistake, a fault is the defect introduced into the software, and a failure is the incorrect behavior observed during execution. Test cases help reveal failures and provide evidence that software behaves as expected. Defect reports further help developers understand, reproduce, prioritize, and fix problems. The exercises show that both normal and fault-based testing are important for achieving reliable software.